

Implement Client Script & UI Policy (Incident)

Phase 5: Project Development Phase

Item	Details
Project Name	Implement Client Script & UI Policy (Incident)
Platform	ServiceNow
Target Table	Incident [incident]
Team ID	SWTID-2026-7176
Team Size	5 members
Development Time	3 hours (3 sprints of 1 hour)

1. Introduction

The Project Development Phase records how the planned solution was built and checked in the ServiceNow instance. It follows the sprint plan from Phase 4 and covers the configuration of one UI Policy, two UI Policy Actions and three Client Scripts on the Incident table, the tests that verified them, and the issues found along the way.

2. Development Approach

- **Environment:** a ServiceNow instance, logged in with administrative access.
- **Order of work:** form controls first (UI Policy and its actions), then automation and validation (Client Scripts), then testing.
- **Team working:** the five members worked in pairs and shared the testing, as set out in the Phase 4 sprint plan.
- **Navigation used:** System UI → UI Policies for the policy and its actions, and System UI → Client Scripts for the three scripts.

3. Implementation

3.1 Task 1: High Impact Control (UI Policy)

The policy applies extra controls only when an incident has High impact, and undoes them when it does not.

Setting	Value
Table	Incident [incident]
Short description (Name)	High Impact Control
Active	True
Condition	Impact is 1 - High
On load	Checked
Reverse if false	Checked
UI Policy Action	Assignment group: Mandatory = True, Visible = Leave alone, Read only = False

3.2 Task 2: UI Policy Action on Urgency

A second action was added to the same policy from its UI Policy Actions related list.

Setting	Value
UI policy	High Impact Control
Field name	Urgency
Read only	True
Visible	Leave alone
Mandatory	Leave alone

3.3 Task 3: onChange Client Script

Setting	Value
Name	Auto set urgency for high impact
Table	Incident [incident]
UI Type / Type	Desktop / onChange
Field name	Impact
Active	True
Isolate script	Checked

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

The script ignores form loads and empty values, then sets Urgency to 1 (High) when Impact changes to 1 (High).

3.4 Task 4: onSubmit Client Script

Setting	Value
Name	Prevent save if Assigned To missing
Table	Incident [incident]
UI Type / Type	Desktop / onSubmit
Active	True
Isolate script	Checked

```
function onSubmit() {
  if (g_form.getValue('impact') == '1' &&
      g_form.getValue('assigned_to') == '') {
    g_form.showErrorMessage(
      'assigned_to',
      'Assigned To is mandatory for High impact incidents.'
    );
    return false;
  }
  return true;
}
```

Returning false stops the save and the error box appears under the Assigned to field.

3.5 Task 5: onCellEdit Client Script

Setting	Value
Name	Prevent state change via list edit
Table	Incident [incident]
UI Type / Type	Desktop / onCellEdit
Field name	State
Active	True
Isolate script	Checked

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
  alert('State cannot be changed from the list. Please open the Incident.');
```

```
// Cancel the list edit
  callback(false, oldValues);
}
```

This version follows the screenshot in the project guide. It shows the alert and cancels the edit, so the State value in the list stays unchanged.

4. Development Log

Sprint	USN	Work Item	Owner	Status
1	USN-1	High Impact Control UI Policy	Member 1	Done
1	USN-2	Assignment group UI Policy Action (Mandatory)	Member 2	Done
1	USN-3	Urgency UI Policy Action (Read only)	Member 3	Done
1	USN-4	onChange script: Auto set urgency	Members 4, 5	Done
2	USN-5	onSubmit script: Prevent save if Assigned To missing	Member 1	Done
2	USN-6	onCellEdit script: Prevent state change via list edit	Members 2, 5	Done
2	USN-7	Mandatory enforcement and successful save tests	Members 3, 4	Done
3	USN-8	Reverse condition test	Members 1, 2	Done
3	USN-9	List edit blocking and form update tests	Members 3, 4	Done

5. Testing and Results

Each configuration was tested with the steps in Task 6 of the project guide. The results below are taken from the guide and its screenshots. Please confirm them against your own runs before you submit.

No.	Test	Steps	Result
T1	Mandatory enforcement	Create a new Incident, set Impact to 1 - High, leave Assigned to empty and click Submit	Pass. The save is blocked and the error "Assigned To is mandatory for High impact incidents." appears under Assigned to. Assignment group is mandatory and Urgency shows 1 - High as read-only.
T2	Successful save	Fill in Assigned to and click Submit	Pass. The Incident saves without errors and Urgency stays 1 - High and read-only.
T3	Reverse condition	Open an Incident with Impact 1 - High and change Impact to 2 - Medium	Pass. The mandatory and read-only rules are lifted. See observation O1 in Section 6.
T4	List edit blocking	In Incident → All, double-click State on any row	Pass. An alert says State cannot be changed from the list and the value remains unchanged.
T5	Form-based update	Open the same Incident, change State on the form and click Update	Pass. The State change saves (State shows In Progress in the list).

6. Observations and Resolutions

No.	Observation	Cause	Resolution
O1	After Impact is changed from High to Medium, Urgency stays at 1 - High but can be edited again	The onChange script only sets Urgency when Impact becomes High and never sets it back; the UI Policy only reverses the read-only setting	Accepted as designed. The user can change Urgency by hand. Resetting it automatically is a future enhancement.
O2	The guide says Assignment group is mandatory, but the save test uses Assigned to	The UI Policy makes Assignment group mandatory, while the onSubmit script checks Assigned to	Both rules were implemented as written, and both were tested in T1.
O3	The onChange script in the guide text differs from its screenshot (addInfoMessage against showFieldMsg)	The guide contains two versions	The text version (addInfoMessage) was used, matching the earlier phases.
O4	The onCellEdit script and alert text in the guide text differ from its screenshot	The guide contains two versions	The screenshot version was used, with callback(false, oldValues).
O5	The rules do not apply to API, import or server-side updates	Client Scripts and UI Policies run only in the browser	Recorded as a limitation. A Business Rule or Data Policy is a possible enhancement.

7. Deliverables

Record	Type	Table
High Impact Control	UI Policy	Incident
Assignment group action	UI Policy Action	Incident
Urgency action	UI Policy Action	Incident
Auto set urgency for high impact	Client Script (onChange)	Incident
Prevent save if Assigned To missing	Client Script (onSubmit)	Incident
Prevent state change via list edit	Client Script (onCellEdit)	Incident

8. Conclusion

All planned items were built within the three sprints. The UI Policy and its two actions control Assignment group and Urgency when Impact is High, the onChange script keeps Urgency consistent, the onSubmit script blocks incomplete high-impact incidents, and the onCellEdit script stops State edits from the list view. All five tests passed. The main observation is that Urgency is not reset when Impact is lowered, and the client-side rules do not cover API or import paths. The next phase covers documentation and demonstration.